Experiment No. 9

Title

Flask-Based Questionnaire Web Application with Template Rendering and Database Integration

Aim

To develop a dynamic web application using Flask by implementing routing, template rendering, form handling, JavaScript validation, and database integration for interactive user profile generation.

Software Required

- Python
- Flask
- SQLite Database
- Text Editor (VS Code / Notepad)
- Web Browser (Chrome / Firefox)

Theory

Flask is a lightweight web framework in Python used to build web applications efficiently. It provides essential features such as routing, request handling, and template rendering, making it suitable for developing dynamic web applications.

A **route** in Flask is used to map a URL to a Python function. When a user accesses a specific URL, the corresponding function is executed, and the response is returned to the browser.

Flask uses the **Jinja2 templating engine**, which allows dynamic content to be embedded within HTML pages. Data from Python can be passed to HTML templates and displayed using template expressions such as {{ variable }}.

In this experiment, a **questionnaire-based application** is developed where users enter personal details such as name, skill, hobby, and goal. The data is processed by Flask and displayed dynamically as a profile.

JavaScript is used for **client-side validation** to ensure that required fields are filled before submitting the form. This improves user experience and reduces invalid data submission.

SQLite is used as a lightweight database to store user data. The application performs basic database operations such as inserting and retrieving records.

The combination of Flask, HTML templates, JavaScript, and database integration enables the development of interactive and user-friendly web applications.

Procedure

1. Install Flask using: pip install flask
2. Create a project folder named experiment9.
3. Create the following files and folders:
    - app.py
    - templates folder
    - static folder
4. Inside templates, create:
    - index.html
    - result.html
    - view.html
5. Inside static, create:
    - style.css
    - script.js
6. Write Flask code in app.py to:
    - Define routes
    - Handle form submission
    - Connect to SQLite database
7. Design HTML templates for questionnaire input and profile display.
8. Add CSS styling for improved user interface.
9. Add JavaScript validation for form input.
10. Run the application:python app.py
11. Open browser and navigate to:http://127.0.0.1:5000/

Program Code

app.py

```
from flask import Flask, render_template, request
import sqlite3
app = Flask(__name__)
# Initialize database
def init_db():
    conn = sqlite3.connect('database.db')
    conn.execute('''
```

```
        CREATE TABLE IF NOT EXISTS profile (
            name TEXT,
            skill TEXT,
            hobby TEXT,
            goal TEXT
        )
    ''')
    conn.close()
init_db()
# Home page
@app.route('/')
def home():
    return render_template("index.html")

# Handle form submission
@app.route('/submit', methods=['POST'])
def submit():
    name = request.form['name']
    skill = request.form['skill']
    hobby = request.form['hobby']
    goal = request.form['goal']
    conn = sqlite3.connect('database.db')
    conn.execute("INSERT INTO profile VALUES (?, ?, ?, ?)",
                 (name, skill, hobby, goal))
    conn.commit()
    conn.close()
    return render_template("result.html", name=name, skill=skill, hobby=hobby, goal=goal)
# View all data
@app.route('/view')
def view():
    conn = sqlite3.connect('database.db')
    data = conn.execute("SELECT * FROM profile").fetchall()
    conn.close()
```

```
    return render_template("view.html", data=data)
if __name__ == '__main__':
    app.run(debug=True)
```

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>About Me</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    <script src="{{ url_for('static', filename='script.js') }}"></script>
</head>
<body>
<div class="container">
    <h1>About Me Questionnaire ✨</h1>
    <form name="myForm" action="/submit" method="POST" onsubmit="return
validateForm()">
        <input type="text" name="name" placeholder="Your Name" required>
        <br>
        <input type="text" name="skill" placeholder="Your Favorite Skill">
        <br>
        <input type="text" name="hobby" placeholder="Your Hobby">
        <br>
        <input type="text" name="goal" placeholder="Your Goal in Life">
        <br>
        <button type="submit">Generate Profile</button>
    </form>
    <br>
    <a href="/view">View All Profiles</a>
</div>
</body>
</html>
```

result.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container">
    <h1>{{ name }}'s Profile 🌟</h1>
    <p><b>Skill:</b> {{ skill }}</p>
    <p><b>Hobby:</b> {{ hobby }}</p>
    <p><b>Goal:</b> {{ goal }}</p>
    <br>
    <a href="/">Create Another</a>
</div>
</body>
</html>
```

view.html

```
<!DOCTYPE html>
<html>
<head>
    <title>All Profiles</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<h1 style="text-align:center; color:white;">All Profiles 🌟</h1>
<div class="cards-container">
    {% for row in data %}
    <div class="card">
```

```
        <h2>{{ row[0] }}</h2>
        <p><b>💡 Skill:</b> {{ row[1] }}</p>
        <p><b>🎯 Hobby:</b> {{ row[2] }}</p>
        <p><b>🚀 Goal:</b> {{ row[3] }}</p>
    </div>
    {% endfor %}
</div>
<br>
<center><a href="/">⬅ Back to Form</a></center>
</body>
</html>
```

style.css

```
/* Background */
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(to right, #4facfe, #00f2fe);
    margin: 0;
}
/* Main Container (Form + Result) */
.container {
    width: 40%;
    margin: 60px auto;
    background: white;
    padding: 30px;
    border-radius: 10px;
    text-align: center;
}
/* Headings */
h1 {
    color: #2563eb;
}

/* Inputs */
```

```
input {
    width: 80%;
    padding: 10px;
    margin: 10px 0;
    border: 1px solid #ccc;
    border-radius: 5px;
}
/* Button */
button {
    padding: 10px 20px;
    background: #2563eb;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}

button:hover {
    background: #1e40af;
}
/* Links */
a {
    text-decoration: none;
    color: #2563eb;
    font-weight: bold;
}
.cards-container {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 20px;
}
.card {
```

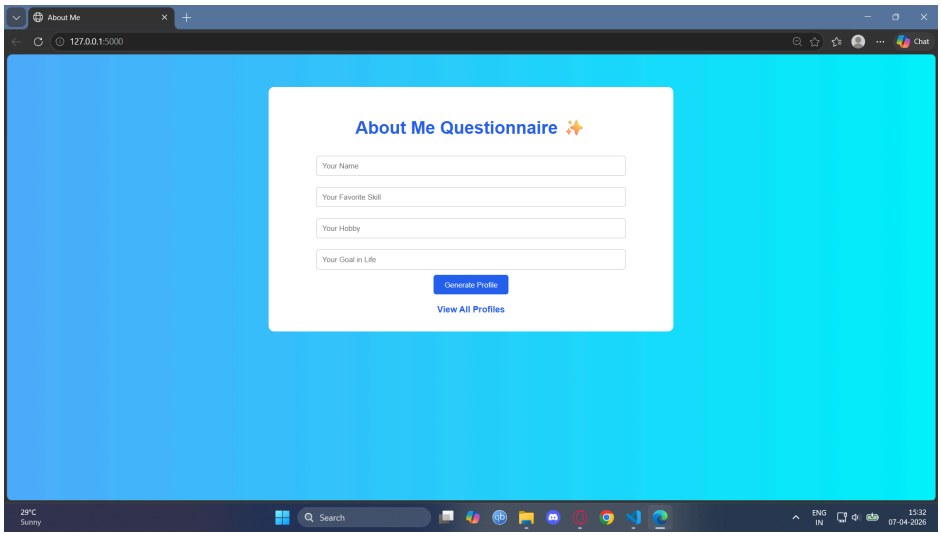
```
    background: white;
    padding: 20px;
    width: 250px;
    border-radius: 12px;
    text-align: center;
    box-shadow: 0 4px 12px rgba(0,0,0,0.2);
    transition: transform 0.3s, box-shadow 0.3s;
}

/* Hover Effect */
.card:hover {
    transform: scale(1.05);
    box-shadow: 0 6px 18px rgba(0,0,0,0.3);
}
.avatar {
    background: #2563eb;
    color: white;
    width: 50px;
    height: 50px;
    border-radius: 50%;
    line-height: 50px;
    margin: 10px auto;
    font-size: 20px;
    font-weight: bold;
}
/* Card Text */
.card p {
    margin: 8px 0;
}
```

script.js

```
function validateForm() {
    let name = document.forms["myForm"]["name"].value;
```

```
    if (name === "") {
        alert("Name is required!");
        return false;
    }
}
```

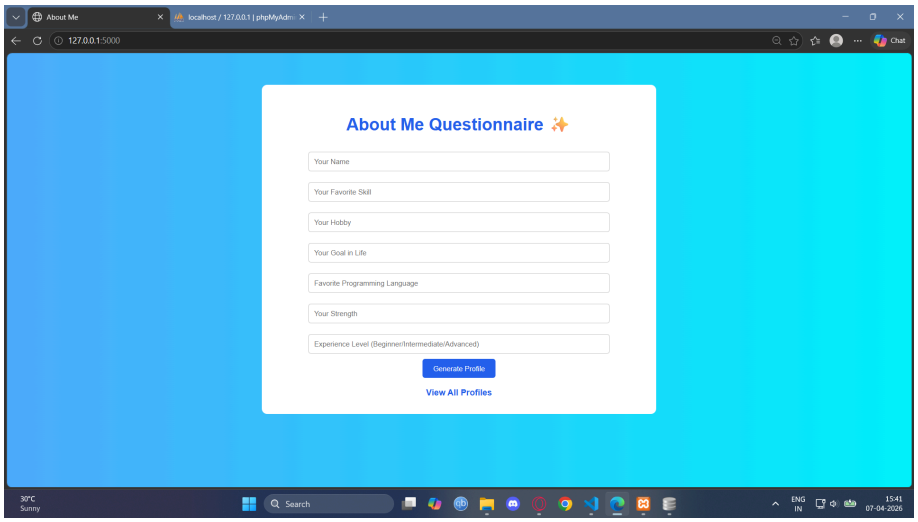


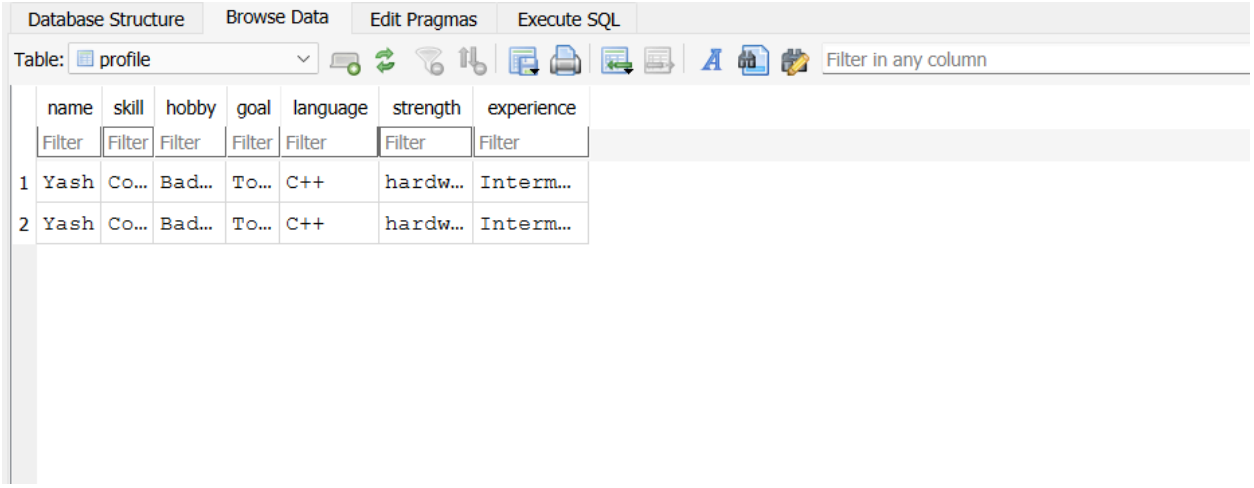
Student Tasks:

1) Modify the Questionnaire Form

Add three additional fields (e.g., *Favorite Language* or *Strength*) in index.html and update:

- app.py (to receive data)
- Database table (add new column)
- result.html (display it)





Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: profile

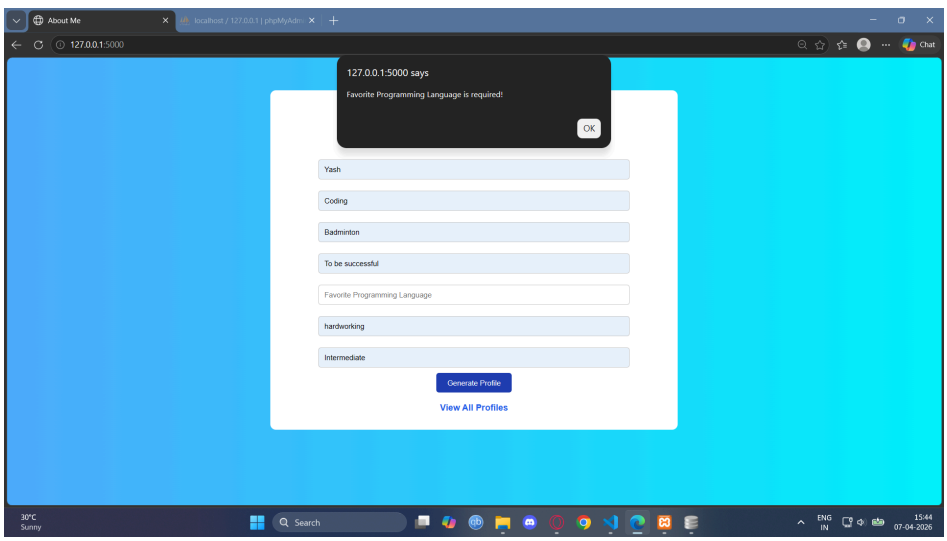
Filter in any column

	name	skill	hobby	goal	language	strength	experience
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	Yash	Co...	Bad...	To...	C++	hardw...	Interm...
2	Yash	Co...	Bad...	To...	C++	hardw...	Interm...

2) Implement Proper Validation

Enhance script.js to:

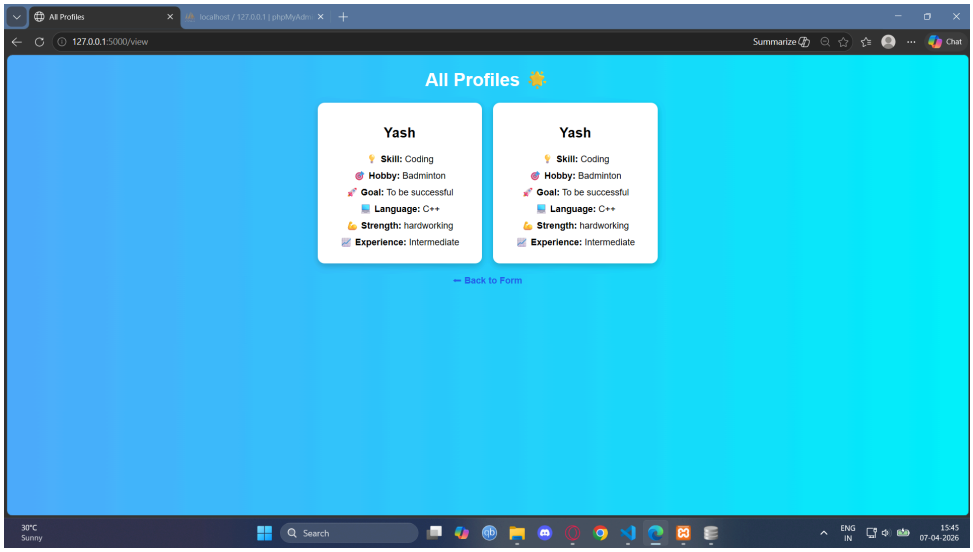
- Validate **all fields** (not just name)
- Show alert if any field is empty



3) Improve Profile Display

Modify result.html to:

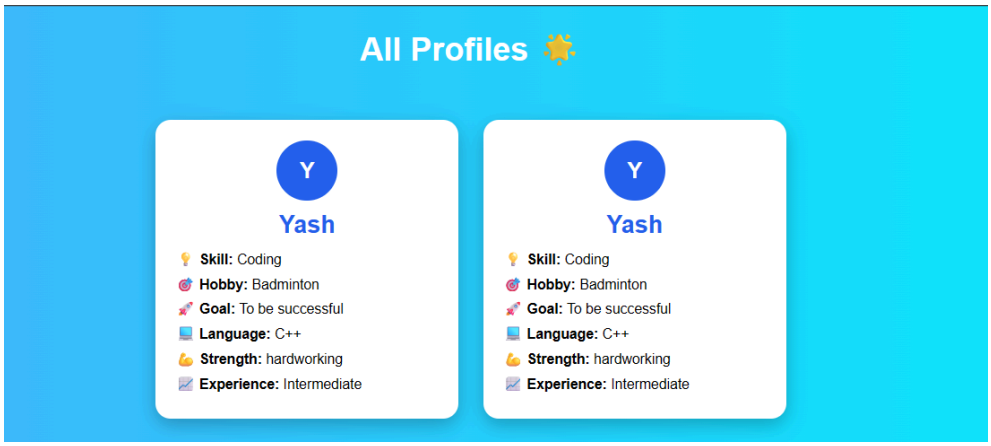
- Add icons/emojis for each field
- Format the output as a **proper profile card layout**



4) Enhance View Page UI

Update view.html by:

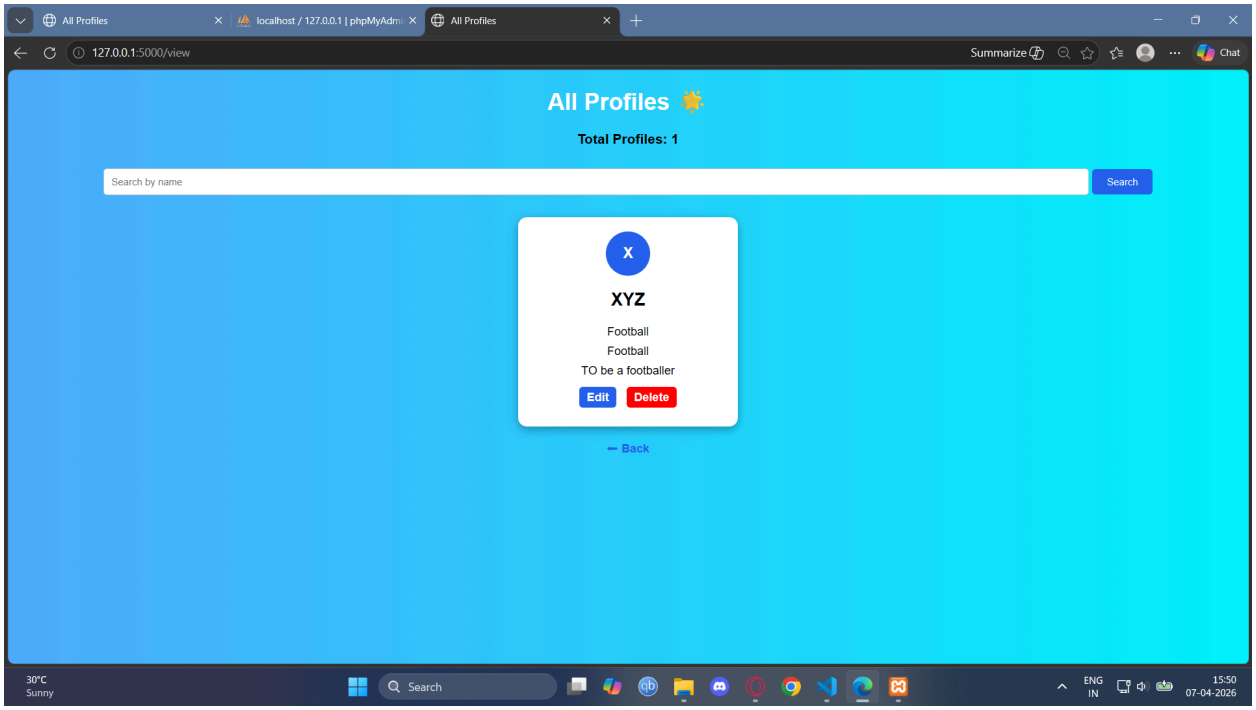
- Adding avatar initials (first letter of name)
- Improving styling using CSS (hover effects, spacing, colors)



5) Add Data Handling Feature

Add a new feature to:

- Count and display total number of profiles stored, add delete record, Update record and search record functionality.


All Profiles
localhost / 127.0.0.1 | phpMyAdmi
All Profiles
127.0.0.1:5000/view
Summarize
Chat
All Profiles
Total Profiles: 1
Search by name
Search
X
XYZ
Football
Football
TO be a footballer
Edit
Delete
Back
30°C
Sunny
Search
ENG
IN
15:50
07-04-2026